

Assignment-5

J.Sreekar Sarma
VU21CSEN0300040

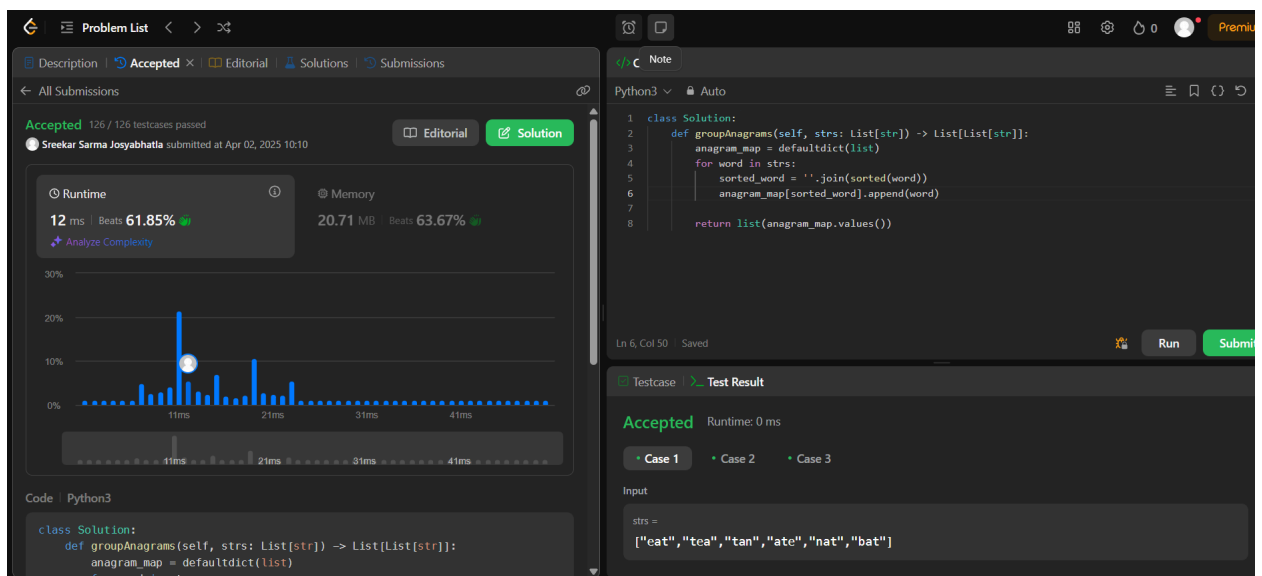
1. Group Anagrams

Python

```
class Solution:
    def groupAnagrams(self, strs: List[str]) -> List[List[str]]:
        anagram_map = defaultdict(list)
        for word in strs:
            sorted_word = ''.join(sorted(word))
            anagram_map[sorted_word].append(word)

        return list(anagram_map.values())
```

OUTPUT



2. Kth Largest Element in an Array

Python

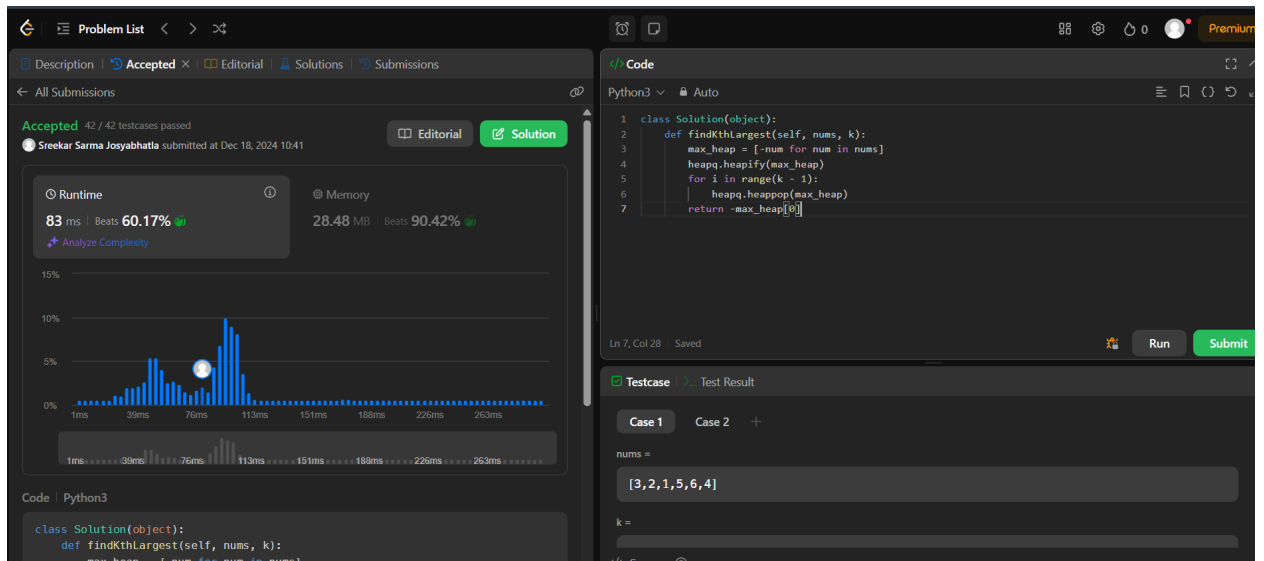
```
class Solution(object):
    def findKthLargest(self, nums, k):
        max_heap = [-num for num in nums]
```

```

heapq.heapify(max_heap)
for i in range(k - 1):
    heapq.heappop(max_heap)
return -max_heap[0]

```

OUTPUT



3. Find Median from Data Stream

Python

```

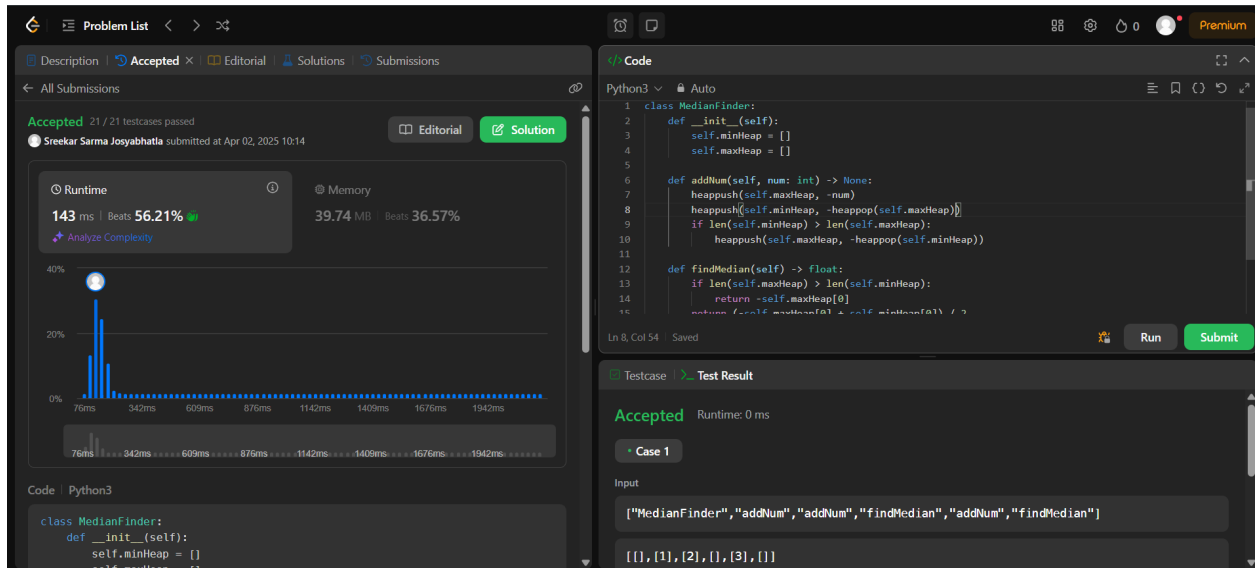
class MedianFinder:
    def __init__(self):
        self.minHeap = []
        self.maxHeap = []

    def addNum(self, num: int) -> None:
        heappush(self.maxHeap, -num)
        heappush(self.minHeap, -heappop(self.maxHeap))
        if len(self.minHeap) > len(self.maxHeap):
            heappush(self.maxHeap, -heappop(self.minHeap))

    def findMedian(self) -> float:
        if len(self.maxHeap) > len(self.minHeap):
            return -self.maxHeap[0]
        return (-self.maxHeap[0] + self.minHeap[0]) / 2

```

OUTPUT



4. Climbing Stairs

Python

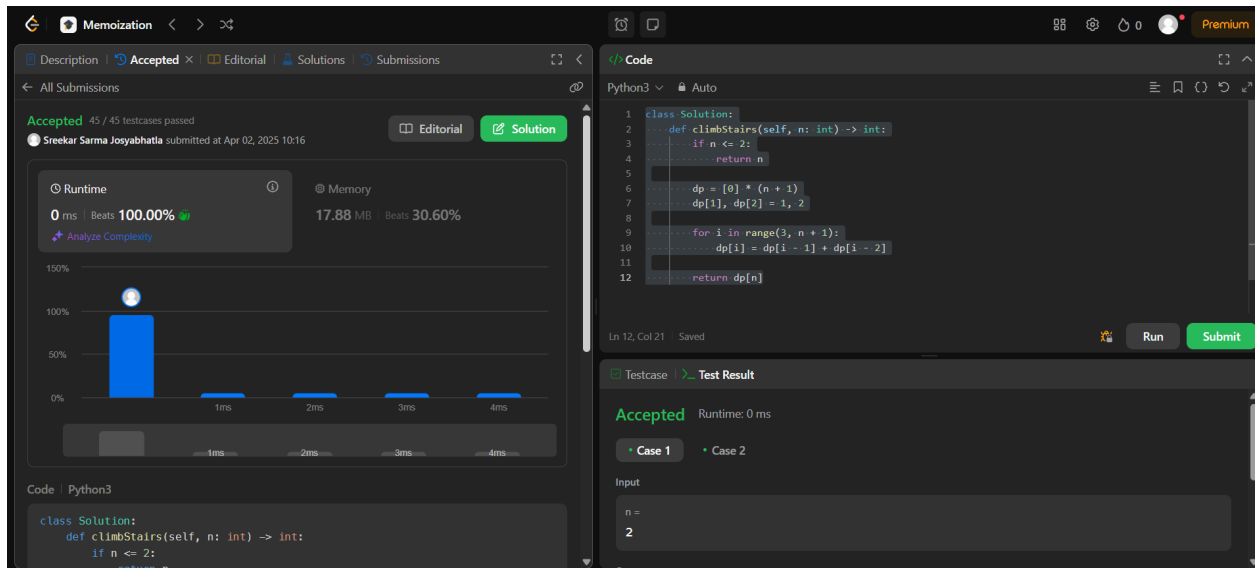
```
class Solution:
    def climbStairs(self, n: int) -> int:
        if n <= 2:
            return n

        dp = [0] * (n + 1)
        dp[1], dp[2] = 1, 2

        for i in range(3, n + 1):
            dp[i] = dp[i - 1] + dp[i - 2]

        return dp[n]
```

OUTPUT



5. Longest Increasing Path in a Matrix

Python

```

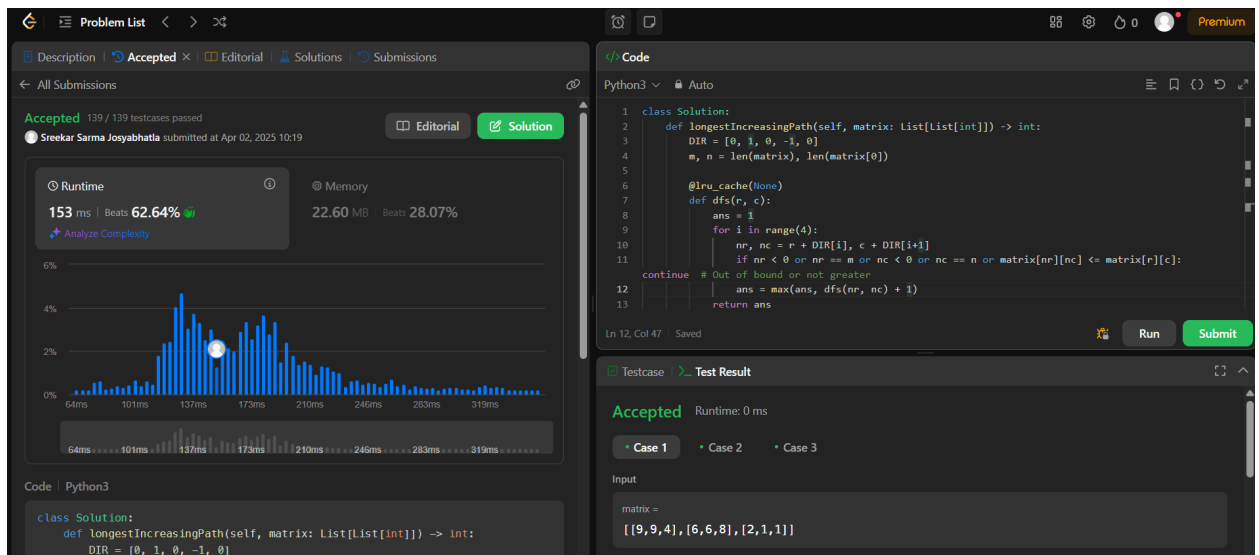
class Solution:
    def longestIncreasingPath(self, matrix: List[List[int]]) -> int:
        DIR = [0, 1, 0, -1, 0]
        m, n = len(matrix), len(matrix[0])

        @lru_cache(None)
        def dfs(r, c):
            ans = 1
            for i in range(4):
                nr, nc = r + DIR[i], c + DIR[i+1]
                if nr < 0 or nr == m or nc < 0 or nc == n or matrix[nr][nc] <= matrix[r][c]: continue # Out of bound or not greater
            ans = max(ans, dfs(nr, nc) + 1)
            return ans

        ans = 0
        for r in range(m):
            for c in range(n):
                ans = max(ans, dfs(r, c))
        return ans

```

OUTPUT :



6. F. Eating Candies

C/C++

```
#include <stdio>
#include <vector>
int main(){

    long t; scanf("%ld", &t);
    while(t--){
        long n; scanf("%ld", &n);
        std::vector<long> w(n); for(long p = 0; p < n; p++){scanf("%ld",
&w[p]);}

        long left(-1), right(n), cnta(0), cntb(0), wa(0), wb(0), ans(0);
        while(cnta + cntb < n){
            if(wa <= wb){++left; ++cnta; wa += w[left];}
            else{--right; ++cntb; wb += w[right];}
            if(wa == wb){ans = cnta + cntb;}
        }

        printf("%ld\n", ans);
    }
}
```

OUTPUT

312521384	Mar/26/2025 10:38 ^{UTC+5.5}	sreekarsarna2003	1674G - Remove Directed Edges	C++17 (GCC 7-32)	Accepted	218 ms	3200 KB
---------------------------	--------------------------------------	------------------	---	------------------	----------	--------	---------